COVERS ALL MAJOR SQL DATABASES

SQL PERFORMANCE EXPLAINED

ENGLISH EDITION



EVERYTHING DEVELOPERS NEED TO KNOW ABOUT SQL PERFORMANCE

MARKUS WINAND

License Agreement

This ebook is licensed for your personal enjoyment only. This ebook may not be re-sold or given away to other people. If you would like to share this book with another person, please purchase an additional copy for each person. If you're reading this book and did not purchase it, or it was not purchased for your use only, then please return to

http://SQL-Performance-Explained.com/

and purchase your own copy. Thank you for respecting the hard work of the author.

This copy is licensed to:

GHEORGHE GABRIEL SICHIM ‹gabriel.sichim@gmail.com›

Publisher:
Markus Winand

Maderspergerstasse 1-3/9/11
1160 Wien
AUSTRIA
<office@winand.at>

Copyright © 2012 Markus Winand

All rights reserved. No part of this publication may be reproduced, stored, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior consent of the publisher.

Many of the names used by manufacturers and sellers to distinguish their products are trademarked. Wherever such designations appear in this book, and we were aware of a trademark claim, the names have been printed in all caps or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and author assume no responsibility for errors and omissions, or for damages resulting from the use of the information contained herein.

The book solely reflects the author's views. The database vendors mentioned have neither supported the work financially nor verified the content.

DGS - Druck- u. Graphikservice GmbH — Wien — Austria

Cover design:
tomasio.design — Mag. Thomas Weninger — Wien — Austria

Cover photo:
Brian Arnold — Turriff — UK

Copy editor:
Nathan Ingvalson — Graz — Austria

2014-08-26

SQL Performance Explained

Everything developers need to know about SQL performance

Markus Winand
Vienna, Austria

Contents

Preface vi

1. Anatomy of an Index 1
    The Index Leaf Nodes 2
    The Search Tree (B-Tree) 4
    Slow Indexes, Part I 6

2. The Where Clause 9
    The Equality Operator 9
        Primary Keys 10
        Concatenated Indexes 12
        Slow Indexes, Part II 18
    Functions 24
        Case-Insensitive Search Using UPPER or LOWER 24
        User-Defined Functions 29
        Over-Indexing 31
    Parameterized Queries 32
    Searching for Ranges 39
        Greater, Less and BETWEEN 39
        Indexing LIKE Filters 45
        Index Merge 49
    Partial Indexes 51
    NULL in the Oracle Database 53
        Indexing NULL 54
        NOT NULL Constraints 56
        Emulating Partial Indexes 60
    Obfuscated Conditions 62
        Date Types 62
        Numeric Strings 68
        Combining Columns 70
        Smart Logic 72
        Math 77

3. Performance and Scalability 79
Performance Impacts of Data Volume 80
Performance Impacts of System Load 85
Response Time and Throughput 87

4. The Join Operation 91
Nested Loops 92
Hash Join 101
Sort Merge 109

5. Clustering Data 111
Index Filter Predicates Used Intentionally 112
Index-Only Scan 116
Index-Organized Tables 122

6. Sorting and Grouping 129
Indexing Order By 130
Indexing `ASC`, `DESC` and `NULLS FIRST`/`LAST` 134
Indexing Group By 139

7. Partial Results 143
Querying Top-N Rows 143
Paging Through Results 147
Using Window Functions for Pagination 156

8. Modifying Data 159
Insert 159
Delete 162
Update 163

A. Execution Plans 165
Oracle Database 166
PostgreSQL 172
SQL Server 180
MySQL 188

Index 193

DEVELOPERS NEED TO INDEX

SQL performance problems are as old as SQL itself—some might even say that SQL is inherently slow. Although this might have been true in the early days of SQL, it is definitely not true anymore. Nevertheless SQL performance problems are still commonplace. How does this happen?

The SQL language is perhaps the most successful fourth-generation programming language (4GL). Its main benefit is the capability to separate *"what"* and *"how"*. An SQL statement is a straight description *what* is needed without instructions as to *how* to get it done. Consider the following example:

```
SELECT date_of_birth
  FROM employees
 WHERE last_name = 'WINAND'
```

The SQL query reads like an English sentence that explains the requested data. Writing SQL statements generally does not require any knowledge about inner workings of the database or the storage system (such as disks, files, etc.). There is no need to tell the database which files to open or how to find the requested rows. Many developers have years of SQL experience yet they know very little about the processing that happens in the database.

The separation of concerns—what is needed versus how to get it—works remarkably well in SQL, but it is still not perfect. The abstraction reaches its limits when it comes to performance: the author of an SQL statement by definition does not care *how* the database executes the statement. Consequently, the author is not responsible for slow execution. However, experience proves the opposite; i.e., the author must know a little bit about the database to prevent performance problems.

It turns out that the only thing *developers* need to learn is how to index. Database indexing is, in fact, a development task. That is because the most important information for proper indexing is not the storage system configuration or the hardware setup. The most important information for indexing is how the application queries the data. This knowledge—about

the access path—is not very accessible to database administrators (DBAs) or external consultants. Quite some time is needed to gather this information through reverse engineering of the application: development, on the other hand, has that information anyway.

This book covers everything developers need to know about indexes—and nothing more. To be more precise, the book covers the most important index type only: the *B-tree index*.

The B-tree index works almost identically in many databases. The book only uses the terminology of the Oracle® database, but the principles apply to other databases as well. Side notes provide relevant information for MySQL, PostgreSQL and SQL Server®.

The structure of the book is tailor-made for developers; most chapters correspond to a particular part of an SQL statement.

CHAPTER 1 - Anatomy of an Index
: The first chapter is the only one that doesn't cover SQL specifically; it is about the fundamental structure of an index. An understanding of the index structure is essential to following the later chapters—don't skip this!

  Although the chapter is rather short—only about eight pages—after working through the chapter you will already understand the phenomenon of slow indexes.

CHAPTER 2 - The Where Clause
: This is where we pull out all the stops. This chapter explains all aspects of the **`where`** clause, from very simple single column lookups to complex clauses for ranges and special cases such as `LIKE`.

  This chapter makes up the main body of the book. Once you learn to use these techniques, you will write much faster SQL.

CHAPTER 3 - Performance and Scalability
: This chapter is a little digression about performance measurements and database scalability. See why adding hardware is not the best solution to slow queries.

CHAPTER 4 - The Join Operation
: Back to SQL: here you will find an explanation of how to use indexes to perform a fast table join.

CHAPTER 5 - Clustering Data

Have you ever wondered if there is any difference between selecting a single column or all columns? Here is the answer—along with a trick to get even better performance.

CHAPTER 6 - Sorting and Grouping

Even `order by` and `group by` can use indexes.

CHAPTER 7 - Partial Results

This chapter explains how to benefit from a "pipelined" execution if you don't need the full result set.

CHAPTER 8 - Insert, Delete and Update

How do indexes affect write performance? Indexes don't come for free—use them wisely!

APPENDIX A - Execution Plans

Asking the database how it executes a statement.

Chapter 1

Anatomy of an Index

"An index makes the query fast" is the most basic explanation of an index I have ever seen. Although it describes the most important aspect of an index very well, it is—unfortunately—not sufficient for this book. This chapter describes the index structure in a less superficial way but doesn't dive too deeply into details. It provides just enough insight for one to understand the SQL performance aspects discussed throughout the book.

An index is a distinct structure in the database that is built using the `create index` statement. It requires its own disk space and holds a copy of the indexed table data. That means that an index is pure redundancy. Creating an index does not change the table data; it just creates a new data structure that refers to the table. A database index is, after all, very much like the index at the end of a book: it occupies its own space, it is highly redundant, and it refers to the actual information stored in a different place.

Clustered Indexes

SQL Server and MySQL (using InnoDB) take a broader view of what "*index*" means. They refer to tables that consist of the index structure only as *clustered indexes*. These tables are called Index-Organized Tables (IOT) in the Oracle database.

Chapter 5, "*Clustering Data*", describes them in more detail and explains their advantages and disadvantages.

Searching in a database index is like searching in a printed telephone directory. The key concept is that all entries are arranged in a well-defined order. Finding data in an ordered data set is fast and easy because the sort order determines each entries position.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

A database index is, however, more complex than a printed directory because it undergoes constant change. Updating a printed directory for every change is impossible for the simple reason that there is no space between existing entries to add new ones. A printed directory bypasses this problem by only handling the accumulated updates with the next printing. An SQL database cannot wait that long. It must process **`insert`**, **`delete`** and **`update`** statements immediately, keeping the index order without moving large amounts of data.

The database combines two data structures to meet the challenge: a doubly linked list and a search tree. These two structures explain most of the database's performance characteristics.

The Index Leaf Nodes

The primary purpose of an index is to provide an ordered representation of the indexed data. It is, however, not possible to store the data sequentially because an **`insert`** statement would need to move the following entries to make room for the new one. Moving large amounts of data is very time-consuming so the **`insert`** statement would be very slow. The solution to the problem is to establish a logical order that is independent of physical order in memory.

The logical order is established via a doubly linked list. Every node has links to two neighboring entries, very much like a chain. New nodes are inserted between two existing nodes by updating their links to refer to the new node. The physical location of the new node doesn't matter because the doubly linked list maintains the logical order.

The data structure is called a *doubly linked list* because each node refers to the preceding and the following node. It enables the database to read the index forwards or backwards as needed. It is thus possible to insert new entries without moving large amounts of data—it just needs to change some pointers.

Doubly linked lists are also used for collections (containers) in many programming languages.

Programming Language	Name
Java	java.util.LinkedList
.NET Framework	System.Collections.Generic.LinkedList
C++	std::list

Databases use doubly linked lists to connect the so-called index *leaf nodes*. Each leaf node is stored in a *database block* or *page*; that is, the database's smallest storage unit. All index blocks are of the same size—typically a few kilobytes. The database uses the space in each block to the extent possible and stores as many index entries as possible in each block. That means that the index order is maintained on two different levels: the index entries within each leaf node, and the leaf nodes among each other using a doubly linked list.

Figure 1.1. Index Leaf Nodes and Corresponding Table Data

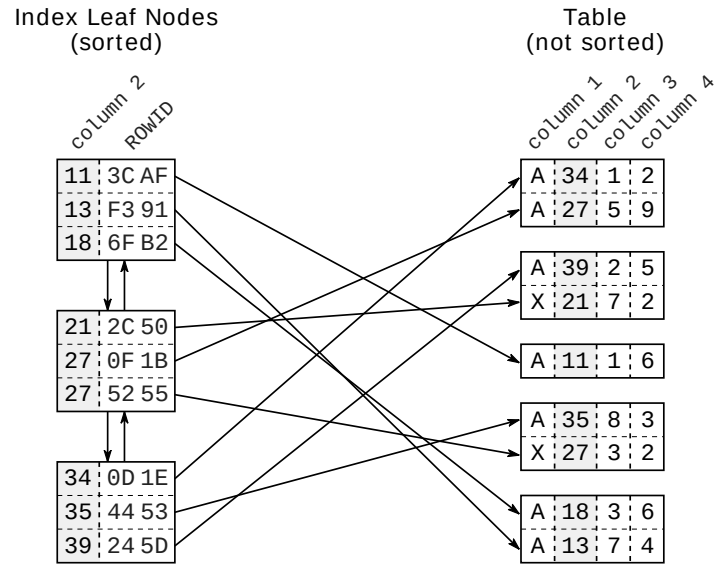


Figure 1.1 illustrates the index leaf nodes and their connection to the table data. Each index entry consists of the indexed columns (the key, column 2) and refers to the corresponding table row (via `ROWID` or `RID`). Unlike the index, the table data is stored in a heap structure and is not sorted at all. There is neither a relationship between the rows stored in the same table block nor is there any connection between the blocks.

The Search Tree (B-Tree)

The index leaf nodes are stored in an arbitrary order—the position on the disk does not correspond to the logical position according to the index order. It is like a telephone directory with shuffled pages. If you search for "Smith" but first open the directory at "Robinson", it is by no means granted that Smith follows Robinson. A database needs a second structure to find the entry among the shuffled pages quickly: a *balanced search tree*—in short: the B-tree.

Figure 1.2. B-tree Structure

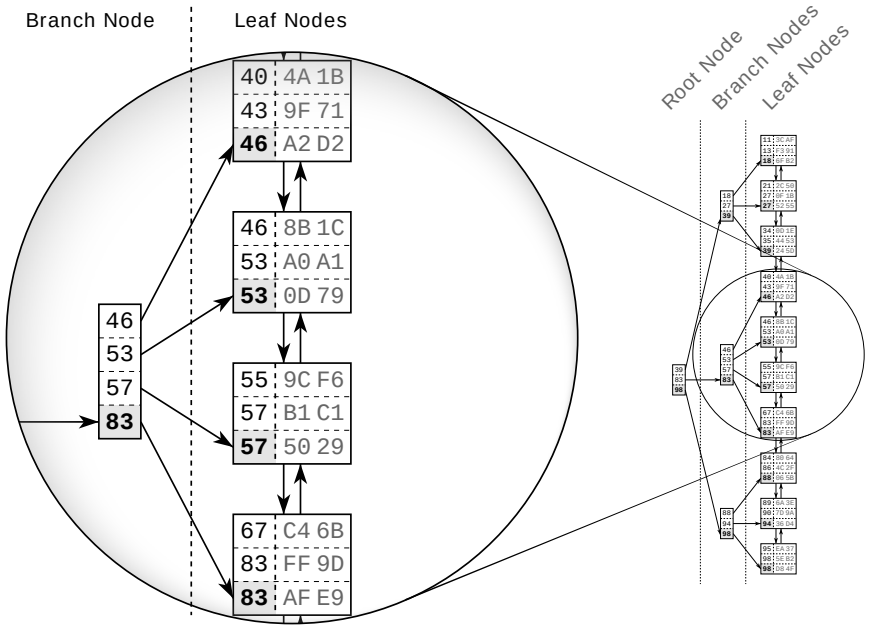


Figure 1.2 shows an example index with 30 entries. The doubly linked list establishes the logical order between the leaf nodes. The root and branch nodes support quick searching among the leaf nodes.

The figure highlights a branch node and the leaf nodes it refers to. Each branch node entry corresponds to the biggest value in the respective leaf node. That is, 46 in the first leaf node so that the first branch node entry is also 46. The same is true for the other leaf nodes so that in the end the

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

branch node has the values 46, 53, 57 and 83. According to this scheme, a branch layer is built up until all the leaf nodes are covered by a branch node.

The next layer is built similarly, but on top of the first branch node level. The procedure repeats until all keys fit into a single node, the *root node*. The structure is a *balanced search tree* because the tree depth is equal at every position; the distance between root node and leaf nodes is the same everywhere.



Note

A B-tree is a balanced tree—not a binary tree.

Once created, the database maintains the index automatically. It applies every **`insert`**, **`delete`** and **`update`** to the index and keeps the tree in balance, thus causing maintenance overhead for write operations. Chapter 8, "*Modifying Data*", explains this in more detail.

Figure 1.3. B-Tree Traversal

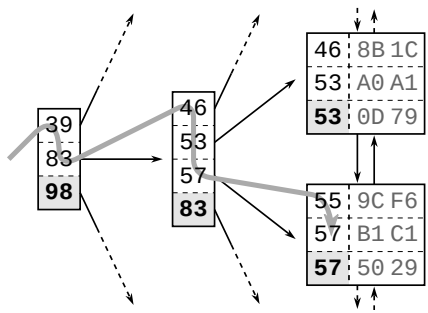


Figure 1.3 shows an index fragment to illustrate a search for the key "57". The tree traversal starts at the root node on the left-hand side. Each entry is processed in ascending order until a value is greater than or equal to (>=) the search term (57). In the figure it is the entry 83. The database follows the reference to the corresponding branch node and repeats the procedure until the tree traversal reaches a leaf node.



Important

The B-tree enables the database to find a leaf node quickly.

The tree traversal is a very efficient operation—so efficient that I refer to it as the *first power of indexing*. It works almost instantly—even on a huge data set. That is primarily because of the tree balance, which allows accessing all elements with the same number of steps, and secondly because of the logarithmic growth of the tree depth. That means that the tree depth grows very slowly compared to the number of leaf nodes. Real world indexes with millions of records have a tree depth of four or five. A tree depth of six is hardly ever seen. The box "Logarithmic Scalability" describes this in more detail.

Slow Indexes, Part I

Despite the efficiency of the tree traversal, there are still cases where an index lookup doesn't work as fast as expected. This contradiction has fueled the myth of the *"degenerated index"* for a long time. The myth proclaims an index rebuild as the miracle solution. The real reason trivial statements can be slow—even when using an index—can be explained on the basis of the previous sections.

The first ingredient for a slow index lookup is the leaf node chain. Consider the search for "57" in Figure 1.3 again. There are obviously two matching entries in the index. At least two entries are the same, to be more precise: the next leaf node could have further entries for "57". The database *must* read the next leaf node to see if there are any more matching entries. That means that an index lookup not only needs to perform the tree traversal, it also needs to follow the leaf node chain.

The second ingredient for a slow index lookup is accessing the table. Even a single leaf node might contain many hits—often hundreds. The corresponding table data is usually scattered across many table blocks (see Figure 1.1, "Index Leaf Nodes and Corresponding Table Data"). That means that there is an additional table access for each hit.

An index lookup requires three steps: (1) the tree traversal; (2) following the leaf node chain; (3) fetching the table data. The tree traversal is the only step that has an upper bound for the number of accessed blocks—the index depth. The other two steps might need to access many blocks—they cause a slow index lookup.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Logarithmic Scalability

In mathematics, the logarithm of a number to a given base is the power or exponent to which the base must be raised in order to produce the number [Wikipedia[1]].

In a search tree the base corresponds to the number of entries per branch node and the exponent to the tree depth. The example index in Figure 1.2 holds up to four entries per node and has a tree depth of three. That means that the index can hold up to 64 (4^3) entries. If it grows by one level, it can already hold 256 entries (4^4). Each time a level is *added*, the maximum number of index entries *quadruples*. The logarithm reverses this function. The tree depth is therefore $\log_4$(number-of-index-entries).

The logarithmic growth enables the example index to search a million records with ten tree levels, but a real world index is even more efficient. The main factor that affects the tree depth, and therefore the lookup performance, is the number of entries in each tree node. This number corresponds to—mathematically speaking—the basis of the logarithm. The higher the basis, the shallower the tree, the faster the traversal.

Tree Depth	Index Entries
3	64
4	256
5	1,024
6	4,096
7	16,384
8	65,536
9	262,144
10	1,048,576

Databases exploit this concept to a maximum extent and put as many entries as possible into each node—often hundreds. That means that every new index level supports a hundred times more entries.

[1] http://en.wikipedia.org/wiki/Logarithm

The origin of the "slow indexes" myth is the misbelief that an index lookup just traverses the tree, hence the idea that a slow index must be caused by a "broken" or "unbalanced" tree. The truth is that you can actually ask most databases how they use an index. The Oracle database is rather verbose in this respect and has three distinct operations that describe a basic index lookup:

INDEX UNIQUE SCAN
: The `INDEX UNIQUE SCAN` performs the tree traversal only. The Oracle database uses this operation if a unique constraint ensures that the search criteria will match no more than one entry.

INDEX RANGE SCAN
: The `INDEX RANGE SCAN` performs the tree traversal *and* follows the leaf node chain to find all matching entries. This is the fallback operation if multiple entries could possibly match the search criteria.

TABLE ACCESS BY INDEX ROWID
: The `TABLE ACCESS BY INDEX ROWID` operation retrieves the row from the table. This operation is (often) performed for every matched record from a preceding index scan operation.

The important point is that an `INDEX RANGE SCAN` can potentially read a large part of an index. If there is one more table access for each row, the query can become slow even when using an index.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 2

The Where Clause

The previous chapter described the structure of indexes and explained the cause of poor index performance. In the next step we learn how to spot and avoid these problems in SQL statements. We start by looking at the **`where`** clause.

The **`where`** clause defines the search condition of an SQL statement, and it thus falls into the core functional domain of an index: finding data quickly. Although the **`where`** clause has a huge impact on performance, it is often phrased carelessly so that the database has to scan a large part of the index. The result: a poorly written **`where`** clause is the first ingredient of a slow query.

This chapter explains how different operators affect index usage and how to make sure that an index is usable for as many queries as possible. The last section shows common anti-patterns and presents alternatives that deliver better performance.

The Equality Operator

The equality operator is both the most trivial and the most frequently used SQL operator. Indexing mistakes that affect performance are still very common and **`where`** clauses that combine multiple conditions are particularly vulnerable.

This section shows how to verify index usage and explains how concatenated indexes can optimize combined conditions. To aid understanding, we will analyze a slow query to see the real world impact of the causes explained in Chapter 1.

Primary Keys

We start with the simplest yet most common **where** clause: the primary key lookup. For the examples throughout this chapter we use the EMPLOYEES table defined as follows:

```
CREATE TABLE employees (
   employee_id   NUMBER         NOT NULL,
   first_name    VARCHAR2(1000) NOT NULL,
   last_name     VARCHAR2(1000) NOT NULL,
   date_of_birth DATE           NOT NULL,
   phone_number  VARCHAR2(1000) NOT NULL,
   CONSTRAINT employees_pk PRIMARY KEY (employee_id)
);
```

The database automatically creates an index for the primary key. That means there is an index on the EMPLOYEE_ID column, even though there is no **create index** statement.

The following query uses the primary key to retrieve an employee's name:

```
SELECT first_name, last_name
  FROM employees
 WHERE employee_id = 123
```

The **where** clause cannot match multiple rows because the primary key constraint ensures uniqueness of the EMPLOYEE_ID values. The database does not need to follow the index leaf nodes—it is enough to traverse the index tree. We can use the so-called *execution plan* for verification:

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    2 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    2 |
|*2 |  INDEX UNIQUE SCAN          | EMPLOYEES_PK |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("EMPLOYEE_ID"=123)
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>